

Q1 RESEARCH PROPOSAL

ASSURE-RAG

Integrity-Gated Retrieval-Augmented Generation for Software Supply Chain Security Triage

Prepared for: Software supply chain security, SBOM, provenance, and secure remediation research

Prepared on: May 11, 2026

Target venues: Journal of Systems and Software, Computers & Security, ACM TOSEM, IEEE Transactions on Software Engineering

Executive Summary

Modern software systems depend on third-party packages, transitive dependencies, automated build pipelines, container images, vulnerability feeds, and generated Software Bills of Materials (SBOMs). These artifacts improve transparency, but they do not automatically produce trustworthy remediation decisions. A generated SBOM may be incomplete, stale, inconsistent with a lockfile, missing transitive dependencies, disconnected from the built artifact, or unsupported by build provenance. A vulnerability advisory may apply to a package name but not to the deployed version, reachable code path, or actual artifact. A large language model can then turn weak or conflicting evidence into confident but unsafe advice.

Core claim: For software supply chain security, semantic relevance is insufficient. RAG systems must verify evidence integrity before recommending dependency upgrades, vulnerability fixes, package removal, or deployment decisions.

ASSURE-RAG proposes an integrity-gated RAG framework for software supply chain security triage. The system retrieves SBOM, manifest, lockfile, vulnerability, package reputation, provenance, and build evidence, but it does not treat retrieved evidence as automatically actionable. Instead, it applies consistency checks and security gates before producing a remediation plan. If evidence conflicts, the system abstains or returns a bounded risk note instead of inventing a fix.

Problem Statement

Existing SBOM and vulnerability management tools are valuable but fragmented. They typically answer narrow questions: which dependencies exist, which CVEs are associated with them, or whether a repository has certain security practices. Developers and security analysts still face the higher-order decision problem:

Can we trust the available evidence enough to act, and what remediation is justified by that evidence?

The gap is visible in four common failure modes:

1. **SBOM inconsistency:** the SBOM does not match package manifests, lockfiles, or build output.
2. **Stale security evidence:** vulnerability data or package metadata is outdated relative to the deployed artifact.
3. **Weak provenance:** source, build, and artifact integrity cannot be established with sufficient confidence.
4. **Unsafe LLM remediation:** a model recommends upgrades or suppressions without verifying applicability, evidence conflicts, or operational risk.

Research Gap

Recent Q1 work has advanced the field in several directions: software supply chain research agendas, SBOM adoption studies, malicious package detection, build reproducibility, and LLM supply chain risk. However, these streams leave a practical gap between evidence transparency and trustworthy remediation. ASSURE-RAG targets that gap by combining evidence retrieval, integrity verification, remediation planning, and final-answer verification in one measurable security workflow.

Existing Research Stream	What It Solves	Remaining Gap Addressed Here
SBOM adoption and quality	Measures whether projects create SBOMs and what fields they contain.	Does not decide whether SBOM evidence is sufficient for safe remediation.
Malicious package detection	Detects suspicious packages in ecosystems such as npm and PyPI.	Does not integrate package risk with SBOM, provenance, and CVE remediation advice.
Reproducible build research	Studies whether artifacts can be rebuilt and verified.	Does not convert build integrity signals into developer-facing triage recommendations.
General RAG and LLM security	Studies retrieved evidence, poisoning, and agent risks.	Does not provide a domain-specific framework for software supply chain remediation.

Proposed Novelty

The proposed novelty is an **integrity-gated RAG architecture** for software supply chain security. Unlike vanilla RAG, which asks whether evidence is relevant, ASSURE-RAG asks whether evidence is relevant, current, consistent, provenance-backed, and safe enough to support a remediation recommendation.

Novelty 1: Evidence Integrity Gates

The system checks consistency among SBOMs, manifests, lockfiles, vulnerability feeds, package metadata, and build/provenance records before allowing evidence into a remediation plan.

Novelty 2: Remediation Planning

Retrieved evidence is separated from actionable advice. The system explicitly classifies recommendations as fix, monitor, investigate, suppress with justification, or abstain.

Novelty 3: Conflict-Aware Abstention

When the SBOM, lockfile, advisory, or provenance evidence conflicts, the system refuses unsupported fixes and explains the missing evidence required for action.

Novelty 4: Security-Specific RAG Benchmark

The proposal introduces SSC-RAGBench, a benchmark for evaluating software supply chain RAG under clean, stale, incomplete, conflicting, and adversarial evidence.

Research Questions

1. **RQ1:** Does integrity-gated RAG reduce unsafe remediation recommendations compared with vanilla RAG and scanner-only baselines?
2. **RQ2:** Can SBOM-manifest-lockfile consistency checks identify stale or incomplete SBOMs with useful precision and recall?
3. **RQ3:** Does adding provenance and package-risk evidence improve vulnerability prioritization and reduce false remediation urgency?
4. **RQ4:** Can the system abstain correctly when vulnerability, package, SBOM, or provenance evidence is conflicting or insufficient?
5. **RQ5:** How much latency and implementation cost does integrity gating add compared with standard RAG?

System Architecture

ASSURE-RAG is designed as a security triage layer over existing repository and package evidence. It does not replace SBOM generators, vulnerability scanners, or build systems. It orchestrates their outputs and verifies whether the evidence supports an action.

```
Repository or package input
-> parse manifest and lockfile
-> load or generate SBOM
-> retrieve vulnerability and advisory evidence
-> retrieve package risk and repository hygiene evidence
-> retrieve provenance and build-integrity evidence
-> run consistency and integrity gates
-> create remediation plan
-> verify final answer against allowed evidence
-> output recommendation, evidence links, risk level, and abstention if needed
```

Key Evidence Types

- **Dependency evidence:** package manifests, lockfiles, direct and transitive dependency graphs.
- **SBOM evidence:** SPDX or CycloneDX documents, component identifiers, versions, relationships, and timestamps.
- **Vulnerability evidence:** CVE, OSV, GitHub Security Advisory, NVD, or ecosystem-specific advisories.
- **Package-risk evidence:** maintainer activity, repository hygiene, install scripts, typosquatting signals, release velocity, and suspicious metadata.
- **Provenance evidence:** SLSA/in-toto attestations, build workflow metadata, signatures, reproducibility signals, and artifact-source links.

Integrity Gates

Gate	Check	Failure Response
SBOM completeness	Required fields and dependency relationships are present.	Mark SBOM as incomplete; avoid high-confidence remediation.
SBOM consistency	SBOM components match manifest and lockfile dependencies.	Flag mismatch; request regeneration or manual validation.
Vulnerability applicability	Advisory package, ecosystem, version range, and fixed version match the actual dependency.	Downgrade confidence or abstain from fix recommendation.
Package integrity	Package metadata, maintainer signals, scripts, and repository hygiene do not indicate high risk.	Classify as investigate or block depending on severity.
Provenance sufficiency	Source, build, artifact, signature, and attestation signals meet a configured assurance threshold.	Do not claim artifact trust; recommend provenance improvement.
Answer verification	Final text mentions only evidence-backed packages, versions, vulnerabilities, and fixes.	Reject answer and return deterministic fallback or abstention.

Benchmark Proposal: SSC-RAGBench

The paper will introduce **SSC-RAGBench**, a benchmark for evaluating RAG systems on software supply chain security triage. The initial benchmark should focus on Python and JavaScript because PyPI and npm have rich dependency ecosystems and known attack patterns.

Benchmark Size and Format

- **Initial study:** 300 labeled cases across Python and JavaScript projects.
- **Extended study:** 1,000 labeled cases if time and annotation capacity allow.
- **Unit of evaluation:** a security query plus repository/package evidence bundle.
- **Labels:** vulnerable package, safe package, required fix, allowed evidence, forbidden claim, must-abstain flag, and expected risk class.

Case Categories

- Direct vulnerable dependency with known fixed version.
- Transitive dependency vulnerability.
- SBOM missing a dependency present in the lockfile.
- SBOM includes a dependency not present in the build input.
- Conflicting vulnerability advisories.
- Package name confusion or typosquatting.
- Suspicious install script or release metadata.
- Missing provenance or unsigned artifact.

- Outdated SBOM timestamp relative to deployed artifact.
- Adversarial package description attempting prompt injection.

Evaluation Design

Baselines

1. **Vanilla LLM:** prompt-only security advice with no retrieval.
2. **Vanilla RAG:** retrieval over advisories and package documents without integrity gates.
3. **Scanner-only baseline:** output from common SBOM or vulnerability scanners.
4. **SBOM-RAG:** RAG over SBOM and advisory evidence, but without consistency gates.
5. **ASSURE-RAG without provenance:** ablation removing build/provenance evidence.
6. **ASSURE-RAG full:** all gates, remediation planner, and final-answer verifier.

Metrics

Metric Family	Metrics
Security correctness	Unsafe recommendation rate, hallucinated fix rate, wrong-version fix rate, vulnerability identification precision.
Evidence integrity	SBOM mismatch detection accuracy, stale SBOM detection accuracy, conflict detection accuracy.
Remediation quality	Correct fix recommendation rate, correct suppress/monitor/investigate classification, evidence coverage.
Abstention	Abstention precision, abstention recall, false abstention rate, false action rate.
Robustness	Attack success rate under poisoned retrieved evidence and prompt-injected package metadata.
Operational cost	Latency p50/p95, token cost, gate runtime, scanner integration overhead.

Expected Contributions

- 1. **A formal integrity-gated RAG architecture** for software supply chain security triage.
- 2. **A remediation planning schema** that separates retrieved evidence from allowed security actions.
- 3. **SSC-RAGBench**, a benchmark for evaluating RAG systems under incomplete, stale, conflicting, and adversarial software supply chain evidence.
- 4. **An empirical comparison** against LLM-only, vanilla RAG, scanner-only, SBOM-RAG, and ablated ASSURE-RAG baselines.
- 5. **Failure analysis** showing where SBOMs, vulnerability feeds, package metadata, and provenance evidence most often fail to support safe remediation.

Implementation Plan

Phase	Work	Output
Weeks 1-2	Define evidence schema, remediation plan schema, and benchmark labels.	Dataset specification and first 50 cases.
Weeks 3-4	Build parsers for manifests, lockfiles, SBOM files, and vulnerability evidence.	Evidence ingestion prototype.
Weeks 5-6	Implement consistency gates and remediation planner.	ASSURE-RAG core pipeline.
Weeks 7-8	Create benchmark cases and run baselines.	Initial results table and error taxonomy.
Weeks 9-10	Run ablations and adversarial evidence experiments.	Security robustness analysis.
Weeks 11-12	Write paper, finalize tables, release replication artifacts.	Q1-ready manuscript draft.

Risk and Scope Control

The main risk is trying to cover the entire software supply chain. That would dilute the contribution. The first paper should focus on dependency and remediation triage for Python and JavaScript projects, with SBOM consistency, vulnerability applicability, package-risk signals, and provenance sufficiency. Container images, firmware, cloud deployment, and enterprise procurement workflows should be treated as future work.

Strategic constraint: Do not pitch this as "AI for supply chain security." Pitch it as **evidence-integrity verification before LLM-assisted software supply chain remediation**. That is narrower, more defensible, and easier to evaluate.

Target Q1 Positioning

- **Journal of Systems and Software:** strongest fit if the contribution emphasizes empirical software engineering, SBOM quality, benchmark design, and developer-facing triage.
- **Computers & Security:** strongest fit if the evaluation emphasizes attack scenarios, poisoned evidence, malicious package metadata, and unsafe recommendation prevention.
- **ACM TOSEM:** highest bar; suitable only if the dataset, taxonomy, and empirical design are very strong.
- **IEEE Transactions on Software Engineering:** suitable if the study deeply connects build provenance, reproducibility, and large-scale empirical analysis.

Reference List

1. Williams, L., Benedetti, G., Hamer, S., Paramitha, R., Rahman, I., Tamanna, M., Tystahl, G., Zahan, N., Morrison, P. J., Acar, Y., Cukier, M., Kaestner, C., Kapravelos, A., Wermke, D., and Enck, W. (2025). Research Directions in Software Supply Chain Security. *ACM Transactions on Software Engineering and Methodology*, 34(5), 1-38. DOI: [10.1145/3714464](https://doi.org/10.1145/3714464).
2. Nocera, S., Romano, S., Di Penta, M., Francese, R., and Scanniello, G. (2025). On the adoption of software bill of materials in open-source software projects. *Journal of Systems and Software*, 230, 112540. DOI: [10.1016/j.jss.2025.112540](https://doi.org/10.1016/j.jss.2025.112540).
3. Wang, S., Zhao, Y., Hou, X., and Wang, H. (2025). Large Language Model Supply Chain: A Research Agenda. *ACM Transactions on Software Engineering and Methodology*. DOI: [10.1145/3708531](https://doi.org/10.1145/3708531).
4. Huang, K. et al. (2024). Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence. *ACM Transactions on Software Engineering and Methodology*. DOI: [10.1145/3705304](https://doi.org/10.1145/3705304).
5. Sharma, A., Baudry, B., and Monperrus, M. (2026). Causes and Canonicalization of Unreproducible Builds in Java. *IEEE Transactions on Software Engineering*, 52(1), 54-69. DOI: [10.1109/TSE.2025.3627891](https://doi.org/10.1109/TSE.2025.3627891).
6. National Institute of Standards and Technology. (2022). Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities. NIST SP 800-218. <https://csrc.nist.gov/pubs/sp/800/218/final>.
7. Cybersecurity and Infrastructure Security Agency. Software Bill of Materials (SBOM). <https://www.cisa.gov/sbom>.
8. SLSA Project. Supply-chain Levels for Software Artifacts. <https://slsa.dev/>.
9. Open Source Security Foundation. OpenSSF Scorecard. <https://openssf.org/scorecard/>.